

**KABALE UNIVERSITY**  
**BCS 3101: Basics of Machine Learning**  
Assignment 2 Companion Guide

# **The Data Pre-processing & Exploratory Data Analysis Pipeline**

*A Step-by-Step Theoretical & Practical Roadmap for Learners*

**Prepared as a shared reference for all learners undertaking  
Assignment 2.**

This guide teaches the concepts, the reasoning behind each technique, and the  
exact sequence of  
actions to follow in your own notebook.

# Table of Contents

---

- 0. How to Use This Guide
- 1. Stage 1: Identify Your Machine Learning Problem & Title
- 2. Stage 2: Source and Describe Your Dataset
- 3. Stage 3: Set Up Your Working Environment
- 4. Stage 4: Import the Required Libraries
- 5. Stage 5: Load the Data and Take a First Look
- 6. Stage 6: Data Pre-processing I: Data Cleaning
- 7. Stage 7: Data Pre-processing II: Data Integration
- 8. Stage 8: Data Pre-processing III: Data Transformation
- 9. Stage 9: Data Pre-processing IV: Reduction, Splitting & Model Readiness
- 10. Stage 10: Exploratory Data Analysis (EDA)
- 11. Stage 11: Writing the Structured Report
- 12. Common Pitfalls That Cost Marks
- 13. Submission Compliance Checklist
- 14. References

## 0. How to Use This Guide

This document is a teaching companion to the official BCS3101 Assignment 2 brief. It does not replace the brief, so read that first for the deadlines, submission mechanics, and academic integrity rules. What this guide adds is the theory, the reasoning, and the exact sequence of actions behind every stage of the pipeline you are required to implement. That way, when you write "I used median imputation because..." in your report, you actually understand why, and you can defend that choice if asked.

Work through the sections below in order. Each stage builds on the one before it: you cannot meaningfully clean data you have not first understood, and you cannot meaningfully scale data you have not first cleaned. Every technique is presented with what it does, why it matters scientifically or mathematically, and a short code pattern you can adapt. Do not copy any of it verbatim without understanding it, since the assignment brief explicitly requires the analysis and write-up to be your own.

The four-stage framing used throughout this guide (cleaning, integration, transformation, and reduction) follows the standard data-mining pipeline taught in the field's leading reference texts, sitting within the broader Knowledge Discovery in Databases (KDD) process of turning raw data into actionable insight. (Han et al., 2011; Fayyad et al., 1996)

### How this guide is organised

Each numbered stage below corresponds to one or more rows in the assignment's marking rubric. The table at the end of Section 0 shows exactly which section to study for each rubric criterion, so you can use this guide both as a learning resource and as a revision checklist before you submit.

### A note on citations

Every technique in this guide that originates from a specific, identifiable piece of published work is cited in-text in (Author, Year) form, the same way you should cite sources in your own report's References section (see the assignment brief's Step 7 and this guide's Section 11). Full bibliographic details for every citation are collected in Section 14, References.

## Rubric-to-Guide Map

Use this table to jump straight to the guidance behind each graded criterion.

| Rubric Criterion                      | Weight | Covered in                                |
|---------------------------------------|--------|-------------------------------------------|
| Pre-processing Pipeline Understanding | 10%    | Sections 1, 6-9 (theory boxes throughout) |
| Dataset Selection & Description       | 10%    | Sections 2 and 3                          |
| Data Cleaning                         | 15%    | Section 6                                 |
| Data Transformation                   | 15%    | Section 8                                 |
| Data Reduction & Splitting            | 10%    | Section 9                                 |

| Rubric Criterion                    | Weight | Covered in |
|-------------------------------------|--------|------------|
| Exploratory Data Analysis           | 20%    | Section 10 |
| Report Structure, Clarity & Writing | 10%    | Section 11 |
| Submission Compliance               | 10%    | Section 13 |

# 1. Stage 1: Identify Your Machine Learning Problem & Title

---

Every pipeline starts with a question, not a dataset. Choosing your dataset before you know what question you want to ask is how students end up with a report that describes data but never frames a real machine learning problem, one of the most heavily penalised gaps in past submissions (it silently drags down Data Description, EDA interpretation, and the Conclusion, because nothing ties back to a purpose).

## 1.1 What Makes a Good Problem Statement

A good machine learning problem statement names three things precisely:

- 1) the target: what you are trying to predict or classify;
- 2) the predictors: what information you will use to predict it; and
- 3) the task type: is this classification (predicting a category, e.g. churn: yes/no), regression (predicting a continuous number, e.g. house price), or an unsupervised task (e.g. clustering customers with no labelled target)? This determines which EDA techniques and which evaluation logic apply later.

| Weak Title (avoid)                | Strong Title (aim for this)                                                           | Why it is stronger                                                                      |
|-----------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| "Analysis of a Dataset"           | "Predicting Student Dropout Risk from Academic and Demographic Records"               | Names the target (dropout), the task (classification), and the predictor domain.        |
| "Machine Learning on Health Data" | "Classifying Diabetes Risk from Clinical Measurements using the Pima Indians Dataset" | Specific dataset, specific target variable, specific task type.                         |
| "Sales Data Exploration"          | "Forecasting Monthly Retail Sales from Historical Transaction and Promotion Data"     | Clarifies this is a regression/time-oriented problem, not just descriptive exploration. |

## 1.2 A Working Checklist Before You Commit

- Can I state the target variable in one sentence, and does it actually exist as a column (or can be derived) in the dataset I plan to use?
- Is this genuinely a classification, regression, or clustering problem, not just "explore the data"?
- Does the problem relate to a real-world decision someone would plausibly want the answer to (a lender, a hospital, a retailer, a school)?
- Am I confident this dataset actually contains enough signal (relevant columns) to investigate this problem meaningfully?

**Why this matters for your grade**

The rubric's Dataset Selection & Description criterion explicitly rewards a "clear, specific, meaningfully investigable ML problem" with the target variable identified. Reports that never name a target variable, or that frame the problem so vaguely it cannot guide later choices (which encoding? which chart?), lose marks here and again later when Target Variable Analysis in EDA has nothing concrete to analyse.

## 2. Stage 2: Source and Describe Your Dataset

### 2.1 Where to Find a Real Dataset

Choose ONE dataset from a free, reputable, publicly accessible repository. The assignment brief names these sources; each is genuinely credible for academic work:

- Kaggle Datasets: [kaggle.com/datasets](https://kaggle.com/datasets) (huge variety, community-vetted, often includes a data dictionary)
- UCI Machine Learning Repository: [archive.ics.uci.edu](https://archive.ics.uci.edu) (classic, well-documented, ideal for classification/regression practice)
- Google Dataset Search: [datasetsearch.research.google.com](https://datasetsearch.research.google.com) (a search engine across many repositories)
- data.gov, data.gov.uk, or the Uganda Bureau of Statistics (UBOS) Open Data Portal (national statistics, strong local relevance)
- World Bank Open Data: [data.worldbank.org](https://data.worldbank.org) (development, economic, and health indicators by country)
- WHO Global Health Observatory: [who.int/data/gho](https://who.int/data/gho) (global health statistics)
- Data.world: [data.world](https://data.world)

### 2.2 The Non-Negotiable Selection Requirements

Before you download anything, verify the dataset satisfies every one of these; a dataset that fails even one of these is graded down under Dataset Selection & Description regardless of how well you process it afterward:

| Requirement                                                            | Why it exists                                                                                                                                                                                                |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Real-world (not a synthetic/toy dataset generated purely for teaching) | Real data has genuine irregularities that force you to make real preprocessing decisions, while a synthetic dataset with no quality problems teaches you nothing about cleaning.                             |
| At least 500 rows and 5 columns                                        | Small samples make summary statistics, correlations, and visualisations unstable and uninformative; five columns give you enough variable types to demonstrate encoding, scaling, and multivariate analysis. |
| Contains genuine data-quality issues (missing values, duplicates,      | This is what the entire Data Cleaning section of your report depends on, since without genuine issues you have nothing authentic to detect and treat.                                                        |

| Requirement                                                            | Why it exists                                                                                  |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| inconsistent formatting, or outliers)                                  |                                                                                                |
| Relates to a machine learning problem you can meaningfully investigate | Ties directly to Stage 1: the dataset must actually be able to answer the question you framed. |

## 2.3 What "Describing" a Dataset Actually Means

A dataset description is not a paragraph of prose about what the columns "sound like"; it is a precise technical profile. At minimum you must state and be able to defend: the dataset's name and a working link to its source; who collected/owns it and, if stated, how and when it was collected; the exact number of rows and columns; a short description of each key feature and its data type; and the target variable, if your problem is supervised.

### Understanding variable types (this shapes every later decision)

Every column falls into one of these types, and the type you assign determines which statistics, encodings, and charts are valid for it later; misclassifying a variable's type is a common root cause of using the wrong technique downstream (e.g. computing a mean of postal codes, or one-hot encoding a truly ordinal variable and losing its order).

| Variable Type                    | Definition                                                       | Valid Summary Stats                             | Example                                                |
|----------------------------------|------------------------------------------------------------------|-------------------------------------------------|--------------------------------------------------------|
| Nominal (categorical, unordered) | Categories with no inherent order                                | Mode, frequency counts                          | Colour, country, payment method                        |
| Ordinal (categorical, ordered)   | Categories with a meaningful order but unequal/undefined spacing | Mode, median, frequency counts                  | Education level, satisfaction rating (Low/Medium/High) |
| Interval (numeric, no true zero) | Numeric, equal spacing, zero is arbitrary                        | Mean, std. dev., median                         | Temperature in °C, calendar year                       |
| Ratio (numeric, true zero)       | Numeric, equal spacing, zero means "none"                        | Mean, std. dev., median, geometric mean, ratios | Age, income, distance, count of items                  |

### Common mistake seen across past submissions

Several past reports named a dataset source in the Data Description section but cited a different, inconsistent source or filename in the References section, or gave row/column counts that did not match what the notebook's own `.shape` output showed. Keep one single source of truth: run `.shape` and record the source link at the moment you load the data, and reuse those exact figures everywhere in your report.

### 3. Stage 3: Set Up Your Working Environment

You may use Jupyter Notebook (local) or Google Colab (cloud). Both produce the same file type (.ipynb) and support the same Python libraries; the choice is about convenience, not correctness.

| Environment                     | How to Start                                                                                                                                                                                            | Advantages                                                                                                                                                               | Trade-offs                                                                                                                                   |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Jupyter Notebook (via Anaconda) | Install Anaconda from <a href="https://anaconda.com">anaconda.com</a> , then launch "Jupyter Notebook" or "JupyterLab" from Anaconda Navigator, or run <code>`jupyter notebook`</code> from a terminal. | Works offline; full control of your Python environment and installed package versions; fast for large datasets stored locally.                                           | Requires a one-time local installation (~3 GB); your machine's own CPU/RAM limits are the ceiling.                                           |
| Jupyter Notebook (via pip)      | <code>`pip install notebook`</code> , then run <code>`jupyter notebook`</code> from a terminal in your project folder.                                                                                  | Lighter-weight than a full Anaconda install if you already have Python.                                                                                                  | You are responsible for installing each library yourself (pandas, numpy, etc.).                                                              |
| Google Colab                    | Go to <a href="https://colab.research.google.com">colab.research.google.com</a> , sign in with a Google account, and create a New Notebook, with nothing to install.                                    | Zero setup; runs in the cloud so it does not depend on your machine's specs; easy to share via a link, which the assignment explicitly allows as your submission method. | Requires an internet connection; free-tier compute/session time is limited; large local files must be uploaded or mounted from Google Drive. |
| VS Code + Jupyter extension     | Install VS Code, then the "Jupyter" and "Python" extensions from the Extensions marketplace; open or create a .ipynb file directly.                                                                     | Full code-editor features (IntelliSense, Git integration, debugging) alongside notebook cells; good if you already use VS Code for other coursework.                     | Slightly more setup than Colab; still needs Python and libraries installed locally or via a configured environment.                          |

#### 3.1 Naming Your Notebook File

Name your file exactly as the brief specifies, before you do anything else, so you never forget:

```
RegistrationNumber_BCS3101_Assignment2.ipynb
# Example: 2024AKCS1234F_BCS3101_Assignment2.ipynb
```

##### Why this matters for your grade

Submission Compliance (10%) explicitly checks the notebook runs end-to-end with visible outputs AND follows this naming convention. A perfect analysis in a file named "Untitled.ipynb" or "assignment2 report.ipynb" is marked down purely for non-compliance, an easily avoidable loss.

### 4. Stage 4: Import the Required Libraries

Import every library you will use in the very first code cell of your notebook, since this is both good practice and lets a reader see your toolset at a glance.

| Library                | Purpose in this pipeline                                                                                                                                      |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pandas                 | (McKinney, 2010) Loading tabular data (CSV/Excel/JSON), inspecting it (.head, .info, .describe), and manipulating it (filtering, grouping, merging).          |
| numpy                  | (Harris et al., 2020) Fast numerical operations and array math that pandas and scikit-learn build on internally (e.g. log transforms, vectorised arithmetic). |
| matplotlib             | The base plotting library: histograms, box plots, scatter plots, line charts.                                                                                 |
| seaborn                | (Waskom, 2021) A statistical plotting layer on top of matplotlib: quicker, better-looking heatmaps, pair plots, and grouped comparisons.                      |
| scikit-learn (sklearn) | (Pedregosa et al., 2011) Pre-processing utilities: train_test_split, StandardScaler / MinMaxScaler, LabelEncoder / OneHotEncoder, and later PCA and imputers. |
| missingno (optional)   | (Bilogur, 2018) Purpose-built visualisations of missing-data patterns (matrix, bar, heatmap, dendrogram).                                                     |
| scipy.stats (optional) | Statistical tests and distributions: e.g. formal skewness/kurtosis, Z-scores, chi-square tests for categorical association.                                   |

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder, OneHotEncoder
from sklearn.decomposition import PCA
from sklearn.impute import SimpleImputer
import missingno as msno      # optional
from scipy import stats      # optional

%matplotlib inline
sns.set_style('whitegrid')
```

## 5. Stage 5: Load the Data and Take a First Look

### 5.1 Loading

```
df = pd.read_csv('your_dataset.csv') # or pd.read_excel(), pd.read_json()
print(f'Rows: {df.shape[0]}, Columns: {df.shape[1]}')
```

### 5.2 Understanding the Dataset Before You Touch It

Before any cleaning decision, you must know what you actually have. Each inspection command below answers a specific question, so run all of them, and interpret the output in a markdown cell rather than just displaying it.

| Command               | Question it Answers                                                  | What to Look For                                                                                                                |
|-----------------------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| df.head() / df.tail() | What do the rows actually look like?                                 | Obvious formatting issues, unexpected text in numeric columns, header misalignment.                                             |
| df.shape              | How much data do I have?                                             | Confirms the $\geq 500$ rows / $\geq 5$ columns requirement.                                                                    |
| df.info()             | What type is each column, and how many non-null values does it have? | Columns typed as 'object' that should be numeric (a sign of embedded text/symbols); quick first read on missingness per column. |
| df.describe()         | What is the numeric distribution of each column?                     | Min/max that look impossible (e.g. negative age), a mean far from the median (skew), a std. dev. of 0 (constant column).        |
| df.dtypes             | Confirms exact data type per column                                  | Whether a column pandas read as int64/float64/object/bool/datetime64 matches what it should logically be.                       |
| df.columns            | What are the exact column names?                                     | Trailing spaces, inconsistent casing, or special characters that will break later code.                                         |
| df.isnull().sum()     | How many missing values, per column?                                 | Feeds directly into Stage 6.1: you need exact counts, not just 'some missing values'.                                           |
| df.duplicated().sum() | How many exact duplicate rows exist?                                 | Feeds directly into Stage 6.2.                                                                                                  |

### Scientific implication

This inspection stage is where you form your hypotheses about the data's quality; you are not yet fixing anything, you are diagnosing. A cleaning decision made without first quantifying the problem ("I imputed missing values" with no stated count) is exactly the kind of thin, unjustified treatment the rubric penalises under Data Cleaning.

## 6. Stage 6: Data Pre-processing I: Data Cleaning

Data cleaning is worth 15% on its own and underpins every later stage: a scaler fitted on data still containing wild outliers, or an encoder applied to a column with inconsistent category spellings, will silently corrupt everything downstream. Treat cleaning as three separate sub-problems: missing values, duplicates, and outliers.

### 6.1 Missing Values

Missing data is not just "absent": statisticians distinguish three mechanisms, and knowing which one plausibly applies to your column should influence your treatment choice. (Rubin, 1976)

| Mechanism                           | Meaning                                                                                   | Example                                                               |
|-------------------------------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| MCAR (Missing Completely at Random) | The chance of a value being missing has nothing to do with any variable, observed or not. | A sensor randomly drops a reading due to a transient hardware glitch. |

| Mechanism                    | Meaning                                                                                         | Example                                                                                                              |
|------------------------------|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| MAR (Missing at Random)      | The chance of missingness depends on other observed variables, not on the missing value itself. | Income is more often missing for younger respondents (age is observed and explains the pattern).                     |
| MNAR (Missing Not at Random) | The chance of missingness depends on the missing value itself.                                  | High earners are less likely to disclose their income, and the missingness depends on the very value that is hidden. |

In practice you cannot always prove which mechanism is at work with certainty, but you should reason about it in your report rather than silently imputing: for example, "missingness in Cabin correlates strongly with lower Fare, suggesting it is not completely random" is exactly the kind of dataset-specific reasoning the rubric rewards over a generic "I filled the missing values." (Little & Rubin, 2019) is a good source if you want to read further on how these mechanisms affect a full statistical analysis, beyond what this guide covers.

### Detecting missing values

```
missing_counts = df.isnull().sum()
missing_pct = (missing_counts / len(df)) * 100
pd.DataFrame({'missing': missing_counts, 'pct': missing_pct.round(2)}).sort_values('pct', ascending=False)
```

### Treatment options and when to use each

| Method                                    | Formula / Logic                                                   | Best suited when...                                                                                                      |
|-------------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Mean imputation                           | $\hat{x} = (1/n) \sum x_i$ (replace missing with the column mean) | The column is roughly symmetric (low skew) and missingness is light.                                                     |
| Median imputation                         | $\hat{x}$ = the middle value when all observed $x_i$ are sorted   | The column is skewed or has outliers, since the median is not pulled by extreme values the way the mean is.              |
| Mode imputation                           | $\hat{x}$ = the most frequently occurring category                | The column is categorical, or a discrete variable with a clearly dominant value.                                         |
| Listwise deletion (drop rows)             | Remove any row containing a missing value                         | Missingness is rare (e.g. <5% of rows) and you can afford to lose those rows without biasing the sample.                 |
| Drop the column entirely                  | Remove the feature                                                | A column is missing in the large majority of rows (e.g. >60-70%) and cannot be meaningfully recovered.                   |
| Model-based imputation (KNN / regression) | Predict the missing value from other correlated features          | You have enough data and correlated predictors to justify a more sophisticated estimate than a single summary statistic. |

```
# Median imputation example
df['age'] = df['age'].fillna(df['age'].median())
```

```
# Mode imputation for a categorical column
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])

# Drop a column that is mostly missing
df = df.drop(columns=['cabin']) # e.g. 77% missing
```

### Scientific implication you must state in your report

Whichever method you choose, name the column, the exact count/percentage missing, the method, and WHY that method suits this specific column's distribution, not a generic "missing values were handled." A histogram of the column before imputing is the fastest way to justify mean vs. median in one sentence.

## 6.2 Duplicate Records

Duplicate rows inflate the apparent importance of whatever pattern they repeat and can leak the same record into both your training and test sets later, artificially improving apparent model performance. Always check, report the count, remove, and (this is the step many reports skip) briefly confirm the removal was appropriate (i.e. these were true accidental duplicates, not legitimately repeated real-world events, such as the same customer making two separate purchases).

```
dup_count = df.duplicated().sum()
print(f'Duplicate rows found: {dup_count}')
df = df.drop_duplicates()
print(f'Shape after removing duplicates: {df.shape}')
```

## 6.3 Outlier Detection and Treatment

An outlier is an observation that lies unusually far from the rest of the data. Outliers matter because they can be genuine (a legitimately rare but important case, e.g. a fraud transaction) or errors (a mistyped age of 999). Your job is to detect them, then make and justify a decision, not merely to notice them.

### Method 1: The IQR (Interquartile Range) Rule

Compute the first quartile (Q1, the 25th percentile) and third quartile (Q3, the 75th percentile) of a numeric column. The interquartile range is: (Tukey, 1977)

$$\text{IQR} = \text{Q3} - \text{Q1}$$

A value is flagged as an outlier if it falls outside the fences:

$$\text{Lower fence} = \text{Q1} - 1.5 \times \text{IQR} \quad \text{Upper fence} = \text{Q3} + 1.5 \times \text{IQR}$$

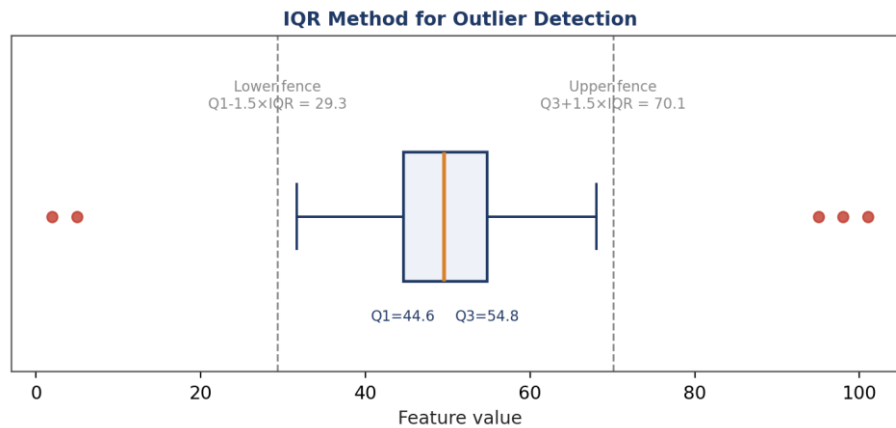


Figure 6.1. The box plot visualises Q1, the median, Q3, and the IQR fences directly; points beyond the fences (red dots) are flagged outliers.

```
Q1 = df['income'].quantile(0.25)
Q3 = df['income'].quantile(0.75)
IQR = Q3 - Q1
lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR
outliers = df[(df['income'] < lower) | (df['income'] > upper)]
print(f'Outliers detected: {len(outliers)}')
```

## Method 2: The Z-score Rule

The Z-score standardises a value against the column's own mean ( $\mu$ ) and standard deviation ( $\sigma$ ):

$$z = (x - \mu) / \sigma$$

A common convention flags any observation with  $|z| > 3$  (i.e. more than three standard deviations from the mean) as an outlier. The Z-score method assumes the column is roughly normally distributed. For heavily skewed columns, the IQR method is generally safer, since Z-scores computed from a skewed mean/std. dev. can mis-flag ordinary values.

```
from scipy import stats
z_scores = stats.zscore(df['income'])
outliers = df[abs(z_scores) > 3]
```

## What to do once you find them

| Treatment                                       | When appropriate                                                                                                                                                |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Remove the rows                                 | Outliers are clearly data-entry errors (e.g. a negative age) with no salvageable information.                                                                   |
| Cap / Winsorise (clip to the fence value)       | You want to retain the row's other information but limit the extreme value's influence on scaling and models.                                                   |
| Log-transform the column                        | The whole distribution is right-skewed with many moderate high values rather than a few extreme errors: a transform tames skew without discarding data.         |
| Retain and flag (add an "is_outlier" indicator) | The extreme values are genuine, important, rare cases (e.g. fraud) that you do not want to remove or distort. The goal is to preserve the signal, not erase it. |

### Common mistake seen across past submissions

Several reports detected outliers with a clear method and count, then never actually treated them, and the same untreated values then reappear, unexplained, in later scaling and EDA sections. If you decide NOT to treat a detected outlier, say so explicitly and justify why (e.g. "the 45 flagged values are genuine high-income earners, retained as valid data"). A stated decision always scores better than a silent gap.

## 7. Stage 7: Data Pre-processing II: Data Integration

Data integration only applies if you are combining more than one data source into a single working dataset (e.g. merging a customer table with a separate transaction table). If your assignment uses a single, already-tabular dataset, you do not need to perform this stage. You must still explicitly say so in your report ("Data integration was not applicable, as this project uses a single unified source file") rather than silently omitting the section, since the rubric rewards an explicitly reasoned judgement over a silent skip.

### If you do combine sources

- Merging/joining: align tables on a shared key column (e.g. `customer_id`) using an inner, left, right, or outer join depending on whether you want only matching records or to preserve all records from one side.
- Resolving schema conflicts: the same real-world concept may have different column names, units, or formats across sources (e.g. 'DOB' vs 'birth\_date'; centimetres vs inches) and must be standardised before joining.
- Entity matching / deduplication across sources: the same real-world entity may appear under slightly different identifiers in each source (e.g. inconsistent spelling of a company name) and must be resolved so it is not double-counted.

```
combined = pd.merge(customers_df, transactions_df, on='customer_id', how='left')
```

## 8. Stage 8: Data Pre-processing III: Data Transformation

Transformation converts your cleaned data into a numeric form and scale that machine learning algorithms can actually consume. Most algorithms cannot use raw text categories, and many are sensitive to features living on very different numeric scales.

### 8.1 Encoding Categorical Variables

Machine learning algorithms operate on numbers, so every categorical (text) column must be converted to a numeric representation before it can be used by most models. The two standard techniques differ in an important assumption:

#### Label Encoding

Assigns each category an integer: e.g. Low  $\rightarrow$  0, Medium  $\rightarrow$  1, High  $\rightarrow$  2. This is mathematically valid ONLY when the categories have a genuine order (ordinal data), because the model will now treat 2 as "twice" 1 in some algorithms, which is meaningless for unordered categories.

**Category  $\rightarrow$  {0, 1, 2, ..., k-1}**

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['education_level_encoded'] = le.fit_transform(df['education_level']) # ordinal: Low/Medium/High
```

### One-Hot Encoding

Creates one new binary (0/1) column per category, so no false order is implied. For a nominal variable with k categories, one-hot encoding produces k new columns; to avoid the "dummy variable trap" (perfect multicollinearity, since the k-th column's value is always fully determined by the other k-1), it is standard practice to drop one column with drop\_first=True.

```
df_encoded = pd.get_dummies(df, columns=['payment_method'], drop_first=True)
# or, using scikit-learn directly:
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(drop='first', sparse_output=False)
```

| Situation                                                                         | Use                                                                                                                                                |
|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Category has a genuine order (Low/Medium/High; ratings 1-5)                       | Label Encoding (or an explicit ordinal map preserving the true order)                                                                              |
| Category has no order, few unique values (e.g. Sex, Region with <10 categories)   | One-Hot Encoding                                                                                                                                   |
| Category has no order but very many unique values (e.g. hundreds of postal codes) | Consider frequency encoding, target encoding, or grouping rare categories first, since one-hot encoding would explode the dataset's dimensionality |

### Common mistake seen across past submissions

A recurring error is label-encoding a nominal (unordered) variable (for example, assigning 0/1/2 to Red/Blue/Green) without acknowledging that this silently implies Green is "greater than" Red. State the nature of each categorical variable (nominal vs. ordinal) explicitly before choosing an encoding, and justify the choice per variable rather than applying one method to every categorical column uniformly.

## 8.2 Feature Scaling: Normalisation and Standardisation

Many algorithms, including anything based on distance (KNN, K-Means, SVM) or gradient descent (logistic regression, neural networks), are sensitive to the scale of each feature. A feature ranging from 0 to 1,000,000 will dominate a feature ranging from 0 to 1 purely because of its scale, not because it is more informative. Scaling puts features on a comparable footing.

### Min-Max Normalisation

Rescales every value into a fixed range, typically [0, 1]:

$$X' = (X - X_{\min}) / (X_{\max} - X_{\min})$$

Best when you know (or can assume) the approximate bounds of the data and want a bounded range, but it is sensitive to outliers, since a single extreme value stretches the whole range and compresses everything else toward one end.

### Z-score Standardisation

Rescales every value to have a mean of 0 and a standard deviation of 1:

$$X' = (X - \mu) / \sigma$$

Does not bound values to a fixed range, but is more robust to outliers than Min-Max and is the conventional choice for algorithms that assume roughly normally distributed, zero-centred input (e.g. PCA, logistic regression, SVM, neural networks).

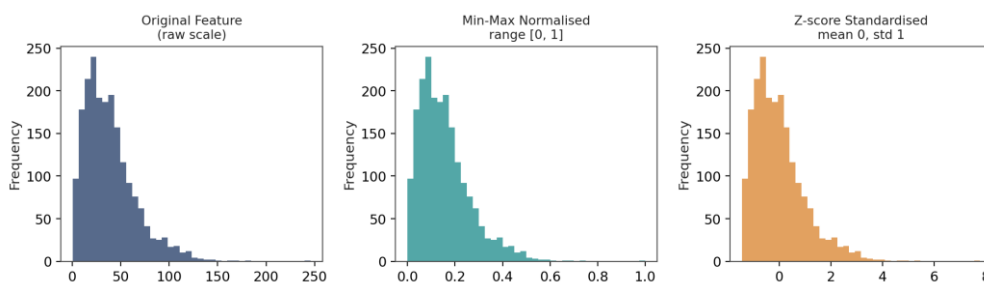


Figure 8.1. The same right-skewed feature before scaling, after Min-Max normalisation (compressed into [0,1]), and after Z-score standardisation (centred at 0). The underlying shape of the distribution does not change; only its scale does.

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# IMPORTANT: fit the scaler on TRAINING data only, then apply it to both sets
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test) # transform only, never re-fit on test data
```

#### Why fitting only on training data matters (data leakage)

If you fit a scaler (or any pre-processing step) on the FULL dataset before splitting into train/test, information from the test set (its mean, its range) leaks into how the training data is scaled. This produces an overly optimistic, dishonest estimate of how well a model would perform on truly unseen data. Always split first, then fit pre-processing steps on the training partition only.

### Why scaling affects convergence: the geometric intuition

Gradient-based training algorithms search for the minimum of a loss function by repeatedly stepping downhill. When features live on very different scales, the loss surface becomes a long, narrow valley (elongated contours) rather than a symmetric bowl. The algorithm must zig-zag back and forth across the narrow axis, taking far more iterations to reach the minimum. Scaling reshapes the loss surface into something closer to circular contours, so gradient descent can step directly toward the minimum, which is exactly why normalised, zero-centred inputs are recommended practice for training neural networks efficiently (LeCun et al., 1998).

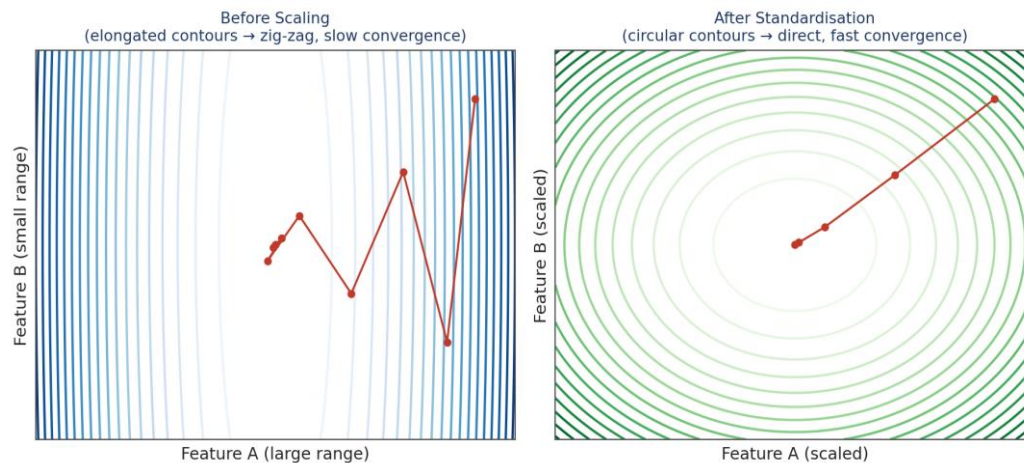


Figure 8.2. Unscaled features produce elongated, narrow loss contours that force an erratic, slow optimisation path (left); after standardisation the contours are roughly circular and the path to the minimum is direct (right).

### Scientific implication for this assignment

Even though this assignment does not require you to train a full model, this is precisely why the rubric's Data Transformation criterion asks you to justify WHY your chosen scaling method suits the data. The justification is not cosmetic, it is about what will make a downstream model converge efficiently and treat every feature fairly.

### Log and Power Transforms

When a numeric feature is heavily right-skewed (a long tail of high values, e.g. income or transaction amounts), a log transform ( $\log(x+1)$ , to safely handle zeros) compresses the tail and makes the distribution closer to symmetric/normal, which benefits both visual interpretation and any model that assumes roughly normal inputs.

```
df['income_log'] = np.log1p(df['income']) # log(1+x) handles zero values safely
```

## 9. Stage 9: Data Pre-processing IV: Reduction, Splitting & Model Readiness

This stage is about efficiency and fairness: reducing unnecessary dimensionality so models train faster and generalise better, correcting class imbalance so a model is not biased toward the majority outcome, and splitting data so your evaluation is honest rather than optimistic.

### 9.1 Feature Selection

Not every column adds useful information. Some are redundant (highly correlated with another feature), irrelevant to the target, or simply noise. Reducing feature count fights the curse of dimensionality: as the number of features grows, the data needed to reliably estimate relationships between them grows exponentially, and models become more prone to overfitting. (Guyon & Elisseeff, 2003; Bellman, 1961)

| Approach         | How it works                                                                                                    | Example technique                                                                                                              |
|------------------|-----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Filter methods   | Score each feature independently of any model, using a statistical measure, then keep the top-scoring features. | Correlation with the target; variance threshold (drop near-constant columns); chi-square test for categorical association.     |
| Wrapper methods  | Repeatedly train a model on different feature subsets and keep the subset that performs best.                   | Recursive Feature Elimination (RFE).                                                                                           |
| Embedded methods | Feature importance is produced as a by-product of training a specific model.                                    | Feature importances from a Random Forest; L1 (Lasso) regularisation, which can shrink irrelevant coefficients to exactly zero. |

```
# Filter example: drop one of any pair of features correlated above 0.9
corr_matrix = df.corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
to_drop = [c for c in upper.columns if any(upper[c] > 0.9)]
df = df.drop(columns=to_drop)
```

## 9.2 Dimensionality Reduction with PCA

Principal Component Analysis (PCA) re-expresses a set of numeric features as a smaller set of new, uncorrelated variables ("principal components"), each capturing as much of the original variance as possible. Mathematically, PCA computes the covariance matrix of the (already standardised) features, finds its eigenvectors and eigenvalues, and projects the data onto the eigenvectors with the largest eigenvalues. Each eigenvalue quantifies how much variance its corresponding component explains. (Pearson, 1901; Hotelling, 1933)

**$\text{Cov}(\mathbf{X}) \mathbf{v} = \lambda \mathbf{v}$     where  $\mathbf{v}$  = eigenvector (a principal component direction) and  $\lambda$  = eigenvalue (variance explained)**

Because PCA is a distance- and variance-based technique, features MUST be standardised (Section 8.2) before applying it, otherwise a feature with a naturally larger numeric range will dominate the components purely due to scale, not genuine variance/importance.

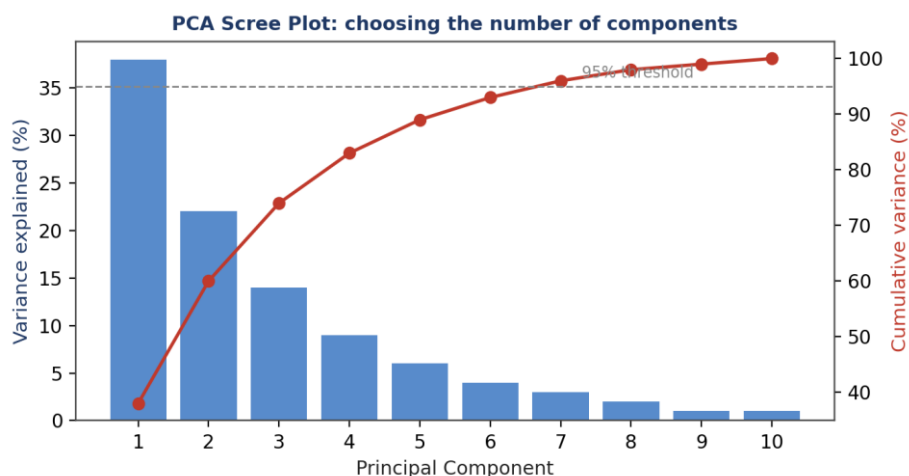


Figure 9.1. A scree plot: bars show variance explained by each individual component, the line shows cumulative variance. Learners typically keep enough components to reach 90-95% cumulative variance (dashed line).

```
from sklearn.decomposition import PCA
pca = PCA(n_components=0.95) # keep enough components to explain 95% of variance
X_reduced = pca.fit_transform(X_scaled)
print(f'Components kept: {pca.n_components_}, variance explained: {pca.explained_variance_ratio_.sum():.2%}')
```

### When to skip PCA, and how to say so

PCA is not always relevant, especially with a modest number of features that are already largely uncorrelated and individually interpretable. If you judge PCA unnecessary, say so explicitly with a reason (e.g. "with only 8 largely independent features, dimensionality reduction was judged unnecessary; feature interpretability was prioritised instead"). The rubric explicitly credits a reasoned decision to omit a step over a silent absence.

## 9.3 Handling Class Imbalance

If your problem is classification and one class vastly outnumbers another (e.g. 90% "no churn" vs. 10% "churn"), a model can achieve high accuracy by simply always predicting the majority class, while being practically useless at the task that actually matters (detecting the minority class). This is why class balance must be examined during EDA (Section 10.6) and, if severe, corrected before or during modelling.

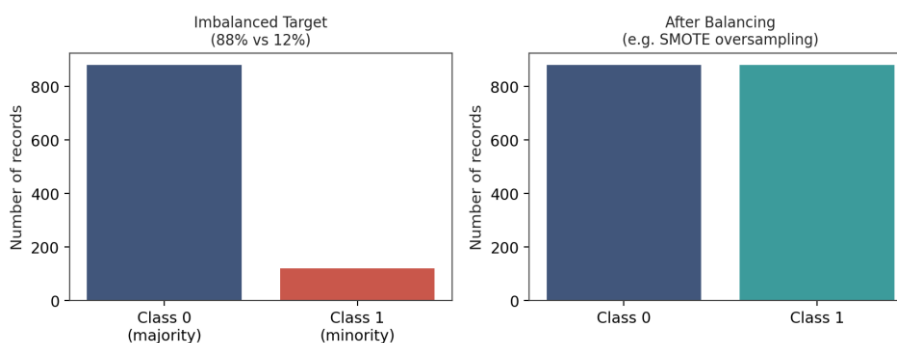


Figure 9.2. An imbalanced target (left) can be corrected by oversampling the minority class (e.g. SMOTE) or undersampling the majority class, producing a more balanced training distribution (right).

| Technique                                          | How it works                                                                                                                                                      |
|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Random oversampling                                | Duplicate existing minority-class rows until classes are balanced.                                                                                                |
| SMOTE (Synthetic Minority Over-sampling Technique) | Generates new synthetic minority-class examples by interpolating between existing minority neighbours, rather than exact duplicates. (Chawla et al., 2002)        |
| Random undersampling                               | Randomly remove majority-class rows until classes are balanced. Simple, but it discards data.                                                                     |
| Class weighting                                    | Instead of changing the data, tell the model (via a <code>class_weight</code> parameter) to penalise mistakes on the minority class more heavily during training. |

| Technique                                                  | How it works                                                                                                                                                              |
|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Stratified sampling (at minimum, in your train/test split) | Ensures the proportion of each class is preserved in both the training and test sets. This is the minimum action every learner should take even without full rebalancing. |

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y # stratify preserves class proportions
)
```

## 9.4 The Train/Test Split

A model must be evaluated on data it has never seen during training, otherwise you are only measuring how well it memorised the training set, not how well it generalises. The standard practice is to split your data before any model-fitting step (and, as emphasised in 8.2, before fitting any scaler or encoder) into a training partition and a held-out test partition. (Kohavi, 1995)

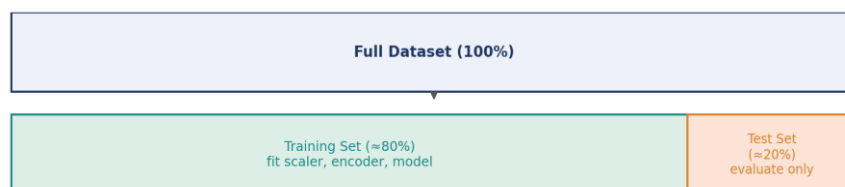


Figure 9.3. A typical 80/20 split: the model learns only from the training portion; the test portion is reserved purely for honest evaluation.

| Parameter              | Purpose                                                                                                                                                                               |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| test_size (e.g. 0.2)   | The proportion of data held out for testing. 70/30 and 80/20 are the most common conventional splits.                                                                                 |
| random_state (e.g. 42) | Fixes the random seed so the split is reproducible. Anyone re-running your notebook gets the exact same split, which is essential for the assignment's "runs end-to-end" requirement. |
| stratify=y             | Forces both partitions to preserve the target variable's class proportions. This is critical whenever the target is imbalanced (Section 9.3) or categorical.                          |

### Common mistake seen across past submissions

A recurring gap is detecting clear class imbalance during EDA, then performing an unstratified split immediately afterward with no acknowledgement of the inconsistency. If you identify imbalance, your split (at minimum) should use stratify=y, and you should explicitly connect the two in your write-up.

## 10. Stage 10: Exploratory Data Analysis (EDA)

EDA is worth 20%, the single largest criterion on the rubric, because it is where you demonstrate genuine understanding of the cleaned data rather than just processing it mechanically. The practice of examining data visually and numerically before formal modelling, rather than jumping straight to a hypothesis test, is the tradition this stage draws on. (Tukey, 1977) The assignment requires at least one technique from each of the six categories below; treat that as a floor, not a ceiling, and always pair every chart or statistic with a written interpretation of what it reveals, not just a caption naming the chart type.

### The single biggest EDA mistake seen across past submissions

Describing a technique in prose ("a correlation heatmap was created to explore relationships") without ever actually embedding the resulting chart or reporting the real numbers it produced. An EDA section is evidence, not narration. Every technique you claim to have used must appear as an embedded chart, table, or specific statistic pulled from your own notebook's output.

## 10.1 Descriptive / Summary Statistics

Before visualising anything, quantify the basic shape of each numeric column.

| Statistic                       | Formula                                                                                         | What it tells you                                                                                             |
|---------------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Mean ( $\mu$ )                  | $\mu = (1/n) \sum x_i$                                                                          | The arithmetic centre of the data. Sensitive to outliers.                                                     |
| Median                          | The middle value of the sorted data (50th percentile)                                           | A robust centre, unaffected by extreme outliers and better for skewed data.                                   |
| Mode                            | The most frequently occurring value                                                             | The most "typical" value: the only valid central-tendency measure for nominal data.                           |
| Variance ( $\sigma^2$ )         | $\sigma^2 = (1/n) \sum (x_i - \mu)^2$                                                           | Average squared deviation from the mean: how spread out the data is.                                          |
| Standard deviation ( $\sigma$ ) | $\sigma = \sqrt{\sigma^2}$                                                                      | Spread in the same units as the original data (unlike variance), easier to interpret directly.                |
| Skewness                        | A standardised third moment; $> 0$ = right-tailed, $< 0$ = left-tailed, $\approx 0$ = symmetric | Tells you whether mean or median better represents the "typical" value, and whether a log transform may help. |
| Quartiles (Q1, Q2, Q3)          | The 25th, 50th, and 75th percentiles                                                            | Describe the spread of the middle 50% of the data; feed directly into the IQR outlier method (Section 6.3).   |

```
df.describe()           # count, mean, std, min, 25%, 50%, 75%, max for every numeric column
df['income'].skew()     # skewness of a single column
df.describe(include='object') # summary for categorical columns: count, unique, top, freq
```

### Interpret, don't just display

Reporting the raw table from `df.describe()` with no comment earns little credit. Instead say, for example: "the mean age (34.2) is close to the median (33.0), suggesting a roughly symmetric distribution, while the maximum recorded age of 187 is almost certainly a data-entry error requiring correction."

## 10.2 Univariate Analysis

Examines one variable at a time to understand its individual distribution. This is essential before looking at relationships between variables.

| Technique                  | Best for              | What to look for                                                                                                 |
|----------------------------|-----------------------|------------------------------------------------------------------------------------------------------------------|
| Histogram                  | Numeric variables     | Shape of the distribution: symmetric, skewed, bimodal (two peaks, possibly signalling two underlying subgroups). |
| Box plot                   | Numeric variables     | Median, spread (IQR), and outliers all in one view. Directly connects to Section 6.3.                            |
| Density / KDE plot         | Numeric variables     | A smoothed version of a histogram, useful for comparing shape across groups on one axis.                         |
| Bar chart / value_counts() | Categorical variables | Which categories dominate, and whether any category is rare enough to need grouping before encoding.             |

```
df['age'].hist(bins=30)
sns.boxplot(x=df['income'])
sns.kdeplot(df['age'])
df['region'].value_counts().plot(kind='bar')
```

## 10.3 Bivariate / Multivariate Analysis

Examines relationships between two or more variables. This is where you discover which features actually relate to your target, and to each other.

The Pearson correlation coefficient (Pearson, 1895) quantifies the strength and direction of a LINEAR relationship between two numeric variables:

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$

$r$  ranges from  $-1$  (perfect negative relationship) to  $+1$  (perfect positive relationship), with  $0$  indicating no linear relationship. Remember: correlation captures only linear association, and correlation is not causation: two variables can be strongly correlated because both are driven by a third, unmeasured factor.

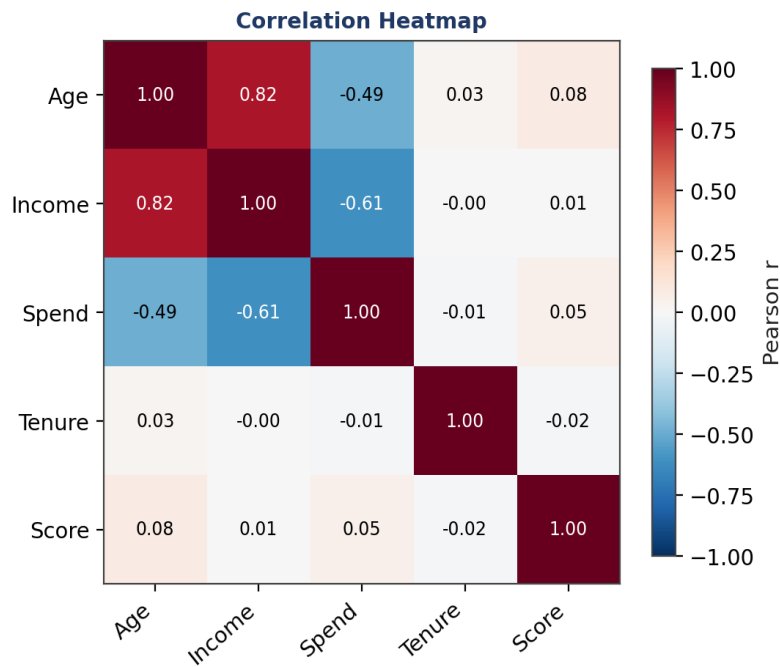


Figure 10.1. A correlation heatmap makes every pairwise Pearson  $r$  immediately visible; dark red/blue cells indicate strong positive/negative relationships worth discussing explicitly.

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap='RdBu_r', vmin=-1, vmax=1)

sns.scatterplot(x='experience_years', y='salary', data=df)
sns.pairplot(df[['age', 'income', 'spend_score']])
pd.crosstab(df['region'], df['churn'], normalize='index') # cross-tabulation for two categorical variables
```

#### A caution repeatedly relevant in past submissions

Watch for correlations that are mathematical artifacts rather than genuine findings. For example, if one column is literally derived from two others ( $\text{Revenue} = \text{Quantity} \times \text{Price}$ ), their strong correlation is guaranteed by arithmetic, not a discovery about customer behaviour. Say so explicitly when it applies; recognising this is itself evidence of genuine understanding.

## 10.4 Missing-Data Visualisation

A visual view of missingness often reveals a PATTERN that a text list of null counts hides, such as whether missing values cluster in the same rows (suggesting a shared cause, such as a survey section that some respondents skipped entirely). The missingno library (Bilogur, 2018) was built specifically for this kind of visual missing-data diagnosis.

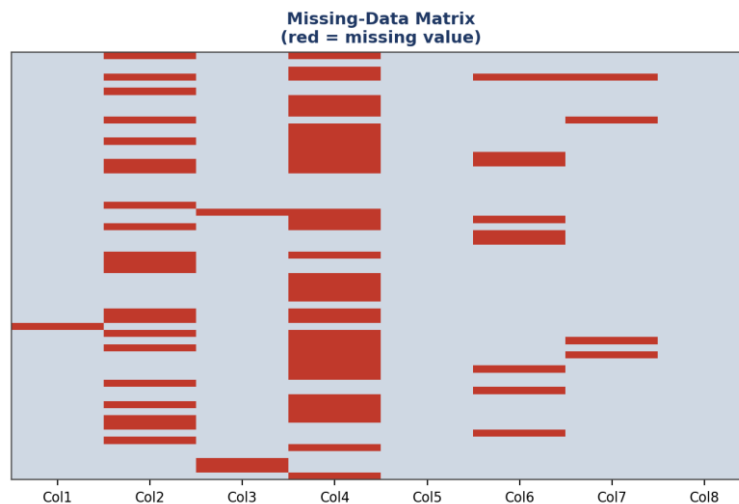


Figure 10.2. A missingness matrix: each red cell is a missing value. Columns 2 and 4 here show heavy, structured missingness rather than scattered random gaps and is worth investigating and reporting.

```
import missingno as msno
msno.matrix(df)
msno.heatmap(df) # shows whether missingness in one column correlates with missingness in another

# Without missingno:
df.isnull().sum().plot(kind='bar')
```

## 10.5 Outlier Detection Visuals (EDA stage)

This is distinct from the outlier TREATMENT you already performed in Data Cleaning (Section 6.3). Here, the goal is to visually confirm and interpret which points are extreme and whether they look like genuine rare cases or residual errors, ideally using a fresh angle (e.g. a scatter plot highlighting flagged points against a second variable) rather than simply reusing the identical box plot from Section 6.3 with no new comment.

```
sns.boxplot(x=df['transaction_amount'])

# Highlight flagged points on a scatter plot
colors = ['red' if v else 'steelblue' for v in df['is_outlier']]
plt.scatter(df['age'], df['income'], c=colors)
```

## 10.6 Target Variable Analysis

Directly examine the variable you are trying to predict (Section 1). For a classification target, chart the class balance; for a regression target, chart its distribution. This is not optional decoration: it determines concrete downstream choices. a badly imbalanced classification target (Section 9.3) demands stratified splitting and possibly resampling; a skewed regression target may benefit from a log transform.

```
df['churn'].value_counts(normalize=True).plot(kind='bar') # class balance
sns.histplot(df['house_price'], kde=True) # regression target distribution
```

**Why this matters for your grade**

The rubric explicitly rewards drawing out the MODELLING IMPLICATIONS of what you found here, for example "the 88%/12% class split means accuracy alone would be a misleading metric, and the train/test split should be stratified (Section 9.4) to preserve this ratio in both partitions." Stating the imbalance without connecting it to a consequence leaves marks on the table.

## 11. Stage 11: Writing the Structured Report

Your notebook is your evidence; your report is where you demonstrate you understand what that evidence means. The report must follow the exact seven-section structure below, be roughly 5-8 pages excluding the cover page, in 11-12pt font, with charts, tables, and short code excerpts embedded directly to support your explanations. Never submit the raw notebook export as a substitute for a written report.

| Section                                  | What It Must Contain                                                                                                                              | Guide Section to Revisit |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| 1. Title                                 | A clear, specific title naming the ML problem you investigated.                                                                                   | Section 1                |
| 2. Overview / Background                 | 1-2 paragraphs: what real-world question this relates to, why it matters, what you hope the data reveals or predicts.                             | Section 1                |
| 3. Data Description                      | Dataset name, source with link, owner/collection method if known, exact row/column counts, key features and their data types, target variable.    | Section 2                |
| 4. Data Preprocessing Techniques Applied | For each stage (cleaning, transformation, reduction, splitting), state what you found, what technique you used, why, and the before/after result. | Sections 6-9             |
| 5. Exploratory Data Analysis             | Describe each EDA technique applied, show the corresponding chart/table, and interpret what it reveals.                                           | Section 10               |
| 6. Conclusion & Next Steps               | One short paragraph reflecting on the data's readiness for modelling and what you would do next.                                                  | -                        |
| 7. References                            | Full citation of the dataset and any other sources consulted, in one consistent style.                                                            | Section 14               |

### 11.1 A Simple Test for Every Paragraph You Write

Before finalising each section, ask: could a reader who has never seen my dataset reconstruct roughly what I did and why, purely from this paragraph and its accompanying evidence? If the answer requires them to trust a claim with no number, chart, or code snippet behind it, the section needs more evidence, not more words.

## 12. Common Pitfalls That Cost Marks

---

The patterns below are drawn from recurring, avoidable weaknesses seen across many past submissions to this exact assignment. None of them require extra work to avoid. They require checking your own report against your own notebook before you submit.

### 12.1 Describing Instead of Demonstrating

The single most common and costly pattern: writing that a technique "was applied" or a chart "was created," without ever embedding the actual chart, table, or code output as evidence. A report that reads well but shows nothing executable is graded as if the work was never done, regardless of how technically correct the prose sounds. The rubric rewards genuine, inspectable evidence over confident narration.

### 12.2 Internal Inconsistency

Figures that do not reconcile across your own report are one of the fastest ways to lose credibility with an examiner: a row count in the Data Description that does not match your notebook's own `.shape` output; a correlation value stated one way in the text and differently in an accompanying table; missing-value counts that shift between sections with no explanation. Before submitting, re-derive every number you have written in prose directly from your notebook's actual output, and fix any mismatch.

### 12.3 Claimed Steps That Were Never Actually Run

Never describe a step you have not actually executed and captured output for, such as writing about a train/test split, a scaler, or a chart that does not appear anywhere in your executed notebook. If a step is genuinely out of scope or not applicable, say so explicitly (see the callout in Section 7 and Section 9.2) rather than describing it as done.

### 12.4 Generic, Templated Justification

A justification that could be pasted into any student's report unchanged ("scaling is important because it helps machine learning algorithms perform better") earns far less credit than one that is anchored in YOUR dataset's actual numbers ("Income ranged from 0 to 2.4 million while Age ranged from 18 to 90; without scaling, Income would dominate any distance-based calculation purely due to its scale"). Whenever you justify a choice, name the actual column, its actual range or distribution, and the actual consequence.

### 12.5 Leftover Placeholders and Unrendered Artifacts

Before submitting, search your report for anything that should never appear in a final document: bracketed placeholder text (e.g. "[insert chart here]", "[INSERT YOUR REGISTRATION NUMBER]"), unrendered markdown or LaTeX symbols, or stray citation markers left over from a reference manager or writing tool. Any of these signal the document was not proofread, and where they replace required evidence, they mean that evidence was never actually produced.

## 12.6 A Report That Is Really a Different Document

Submitting the wrong file, an old draft, a generic slide deck, or a duplicate of a different assignment entirely is a real and costly risk, especially when working across multiple courses or resending after a technical issue. Before you attach anything to your submission email, open the exact file you are about to send and confirm it is your current, complete Assignment 2 report for THIS dataset. This single check has been the difference between a strong grade and a zero in past cohorts.

## 12.7 Length and Padding vs. Substance

Both directions are penalised: a report far under 5 pages is unlikely to contain enough genuine evidence across all seven required sections, while a report that balloons past 8 pages by repeating the same chart with minimal new text, or by leaving excessive white space around oversized images, dilutes rather than strengthens the analysis. Aim for density of evidence, not page count.

## 12.8 Data Legitimacy

A dataset explicitly described as "synthetic" or "realistic simulated data" with no traceable source violates the assignment's core requirement for a real-world dataset with genuine quality issues, however well the rest of the pipeline is executed. Confirm your dataset's provenance is real and citable BEFORE you invest hours of work in it (Section 2.2).

# 13. Submission Compliance Checklist

---

Submission Compliance is worth 10% and is entirely within your control. Every item below is a yes/no check you can perform yourself before the deadline.

- My notebook is named exactly `RegistrationNumber_BCS3101_Assignment2.ipynb`.
- Every cell in my notebook has been run top-to-bottom in a fresh kernel/runtime (Kernel → Restart & Run All, or Runtime → Run all in Colab), with every output, chart, and print statement visible.
- My report is a genuinely written document (not the raw notebook exported as a substitute), 5-8 pages excluding the cover page, 11-12pt font, saved/exported as a PDF.
- My report contains all seven required sections, in the required order, each with the content specified in Section 11's table above.
- Every number stated in my report matches the corresponding output in my notebook exactly.
- My References section cites my dataset with source, owner/publisher, year accessed, and a working link.
- I have written one email to [ak2.acleophas@gmail.com](mailto:ak2.acleophas@gmail.com), before the deadline, with the exact subject line: "BCS3101 Assignment 2 – [Your Full Name] – [Your Registration Number]".
- That email body states my full name and registration number, my ML problem title, and my dataset name/source link.

- That email includes my notebook: either a Google Colab link set to "Anyone with the link can view", or the .ipynb file attached with all cells already executed.
- That email includes my written report attached as a PDF.
- I have also submitted to the university e-learning platform, as required in addition to the email.
- I opened the exact files I am about to submit one final time to confirm they are current, complete, and belong to this assignment.

### On academic integrity

This is an individual assignment: your notebook and report must reflect your own data-handling decisions and your own understanding. You may consult documentation or AI tools to debug code, but the analysis, interpretation, and write-up must be yours, not generated on your behalf. Reusing a short public code snippet is acceptable only if you understand it and cite the source in a comment. Submissions found to be plagiarised or copied from another learner receive a mark of zero and may be referred for disciplinary review.

## 14. References

---

Every source cited in-text throughout this guide is listed below in full, in APA 7th-edition style, alphabetically by author surname. Each was checked directly against its publisher, journal, or official project page before inclusion. Two entries carry a short note where one secondary detail (an exact page number in a large proceedings volume, in both cases) could not be independently confirmed against the primary source; the rest of each citation is fully verified.

- Bellman, R. (1961). *Adaptive control processes: A guided tour*. Princeton University Press.
- Bilogur, A. (2018). Missingno: A missing data visualization suite. *Journal of Open Source Software*, 3(22), 547. <https://doi.org/10.21105/joss.00547>
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>
- Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). From data mining to knowledge discovery in databases. *AI Magazine*, 17(3), 37–54. <https://doi.org/10.1609/aimag.v17i3.1230>
- Guyon, I., & Elisseeff, A. (2003). An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3, 1157–1182.
- Han, J., Kamber, M., & Pei, J. (2011). *Data mining: Concepts and techniques* (3rd ed.). Morgan Kaufmann/Elsevier.
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ...

- Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24, 417–441. <https://doi.org/10.1037/h0071325>
- Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence* (Vol. 2, pp. 1137–1145). Morgan Kaufmann.
- Note: The proceedings page range appears as 1137–1145 in most secondary listings and Google Scholar's record; a minority of sources give 1137–1143. Verify against a library copy if an exact page citation is required.*
- LeCun, Y., Bottou, L., Orr, G. B., & Müller, K.-R. (1998). Efficient backprop. In G. B. Orr & K.-R. Müller (Eds.), *Neural networks: Tricks of the trade* (Lecture Notes in Computer Science, Vol. 1524, pp. 9–50). Springer.
- Little, R. J. A., & Rubin, D. B. (2019). *Statistical analysis with missing data* (3rd ed.). Wiley.
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.
- Pearson, K. (1895). Note on regression and inheritance in the case of two parents. *Proceedings of the Royal Society of London*, 58, 240–242. <https://doi.org/10.1098/rspl.1895.0041>
- Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, Series 6, 2(11), 559–572. <https://doi.org/10.1080/14786440109462720>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3), 581–592. <https://doi.org/10.1093/biomet/63.3.581>
- Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.
- Note: This work is the standard attribution for the box plot and the exploratory-data-analysis tradition generally; the specific 1.5×IQR outlier-fence convention is universally taught as originating here, though an exact page citation for that convention could not be independently verified.*
- Waskom, M. L. (2021). seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>

## Further Reading (official documentation, not separately cited in-text)

Beyond the cited academic and software sources above, the following official documentation pages are useful working references while you build your own notebook. These are living web

pages rather than fixed publications, so no formal citation applies, but they are the authoritative source for the tools this guide teaches.

- Kaggle Datasets, at [kaggle.com/datasets](https://kaggle.com/datasets), for sourcing your dataset and browsing example community notebooks. Use these for inspiration on technique only, never for copying analysis.
- UCI Machine Learning Repository, at [archive.ics.uci.edu](https://archive.ics.uci.edu).
- Scikit-learn User Guide, at [scikit-learn.org/stable/user\\_guide.html](https://scikit-learn.org/stable/user_guide.html): official documentation for every preprocessing utility referenced in Sections 8 and 9.
- Pandas documentation, at [pandas.pydata.org/docs](https://pandas.pydata.org/docs): official reference for every DataFrame method referenced in Sections 5 to 7.
- Seaborn example gallery, at [seaborn.pydata.org/examples](https://seaborn.pydata.org/examples): visual reference for every chart type referenced in Section 10.